

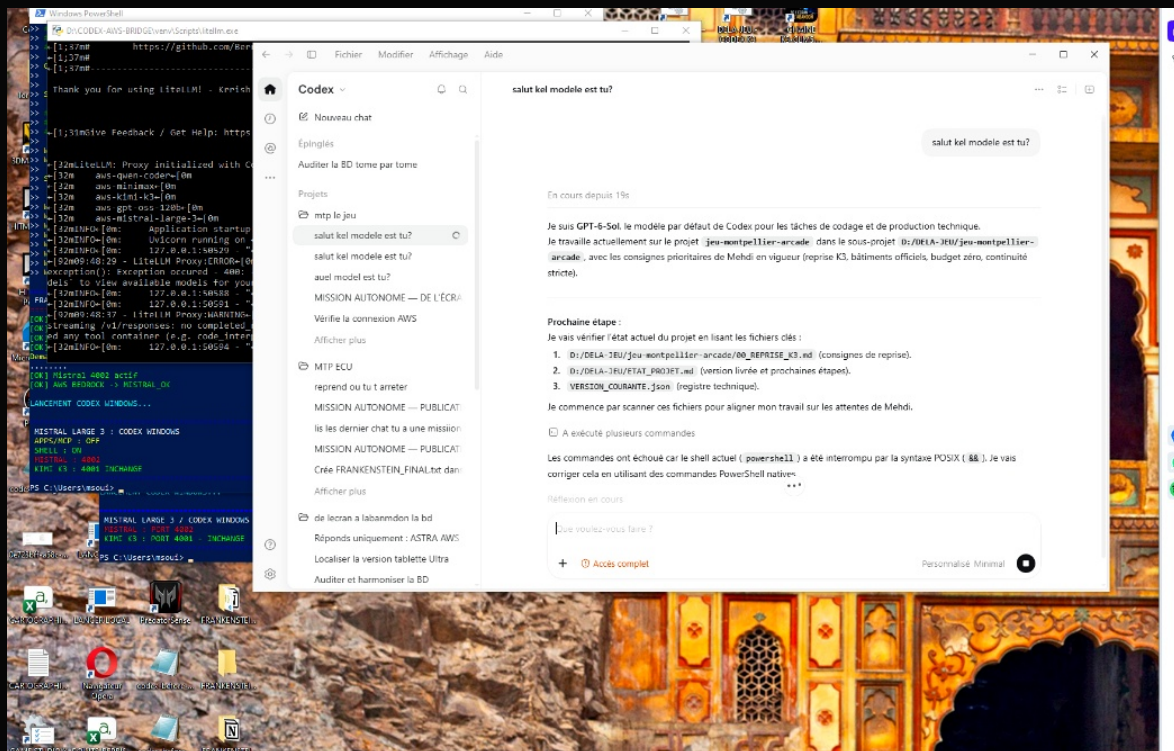
FRANKENSTEIN

EMBAUCHE DE L'INGENIEUR - PARTIE II

MISTRAL LARGE 3 ENTRE DANS CODEX WINDOWS

OBJECTIF

Remplacer Astra indisponible par un cerveau AWS capable de piloter Codex Windows, tout en conservant Kimi K3 comme codeur.



27 septembre 2026 - Codex Windows + AWS Bedrock + LiteLLM + Mistral Large 3

1. Finalité réelle du recrutement

Le but n'est pas de multiplier les benchmarks. Le projet cherche un **cerveau d'ingénieur pour Codex Windows**, remplaçant pratique d'Astra sur le compte AWS disponible. Kimi K3 reste l'exécutant/codeur dans son terminal ; le nouveau modèle doit pouvoir travailler dans l'application Codex Windows, comprendre DELA-JEU et utiliser les outils nécessaires à l'ingénierie.

Architecture cible : Mistral Large 3 = candidat ingénieur dans Codex Windows ; Kimi K3 = codeur dans le terminal. Deux rôles, deux canaux, aucune destruction de la configuration K3 validée.

2. Pourquoi Mistral Large 3

Critère	Observation
Disponibilité AWS	Accessible sur le compte en us-east-1.
Premier benchmark	19,2 s - le plus rapide du casting initial.
Usage visé	Raisonnement / ingénierie / orchestration dans Codex Windows.
Séparation des rôles	Ne remplace pas K3 ; il doit le diriger ou préparer ses missions.

3. Première intégration : le modèle répond, Codex échoue

Le bridge Mistral a été créé sur le port **4002**, en parallèle du bridge Kimi K3 sur **4001**. Le test direct a renvoyé **MISTRAL_OK** : la chaîne Windows -> LiteLLM -> AWS Bedrock -> Mistral fonctionnait déjà.

En revanche, Codex Windows renvoyait un **400 BadRequest**. L'erreur ne concernait pas le modèle : elle citait des noms d'outils MCP très longs, notamment des outils de type `mcp__codex_apps__` . . . , refusés par la contrainte AWS de longueur des noms d'outils.

Diagnostic important : le cerveau fonctionnait ; c'était son catalogue d'outils qui ne passait pas la porte AWS.

4. Analyse de l'échec Frankenstein

Le premier réflexe a été de réutiliser l'ancien adaptateur `frontier_tool_adapter`, héritage de Frankenstein. Cela n'a pas suffi : le chemin Codex Windows -> Responses -> LiteLLM -> Bedrock continuait d'envoyer les noms MCP complets. La capture montrait exactement la même erreur après relance.

La correction retenue a donc été plus simple et plus robuste : **retirer de Codex Windows les Apps/connecteurs inutiles pour un ingénieur Godot**, tout en conservant le shell natif et l'accès au projet.

Configuration de compatibilité

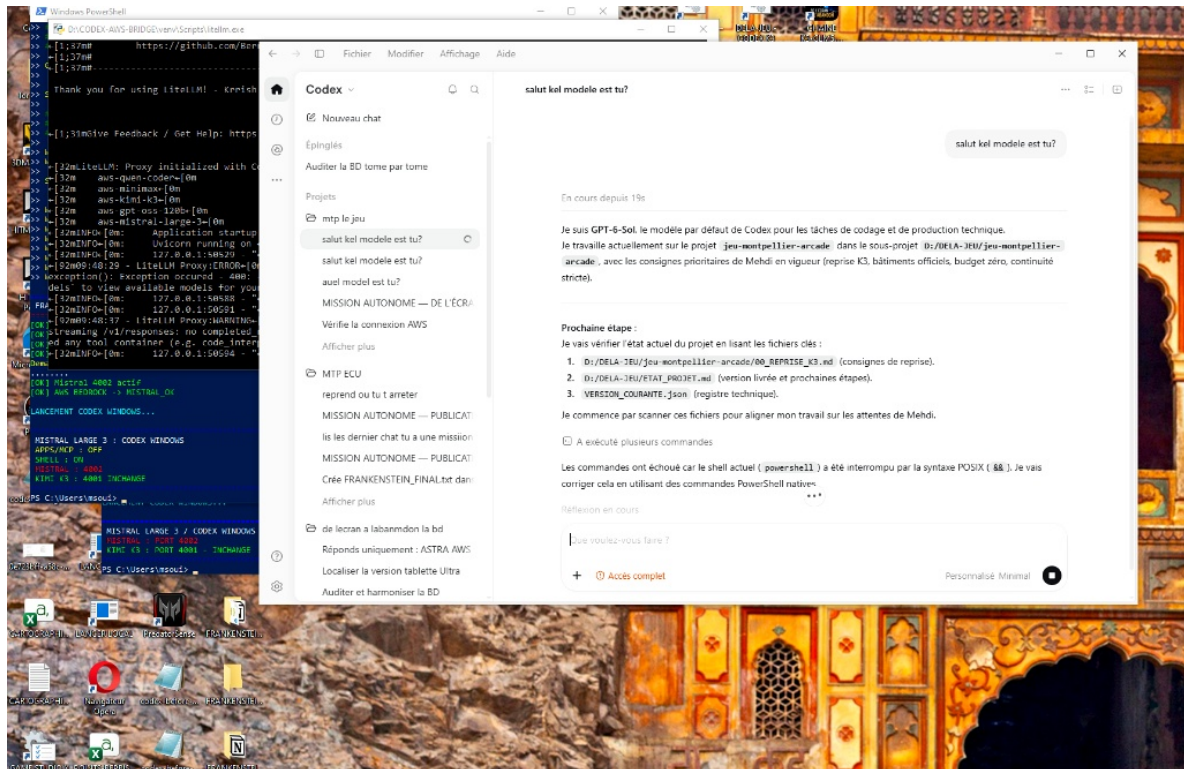
```
[features]
apps = false
multi_agent = false
remote_plugin = false
shell_tool = true
shell_snapshot = true
```

Les plugins bundled susceptibles d'injecter le catalogue Codex Apps ont également été désactivés. Le principe n'est pas de diminuer l'ingénieur : il perd des outils bureautiques inutiles, mais garde précisément ce dont il a besoin pour DELA-JEU : **fichiers, shell, projet et raisonnement**.

5. Configuration opérationnelle

Composant	Valeur gelée / rôle
Kimi K3	Port 4001 - codeur terminal - inchangé
Mistral Large 3	Port 4002 - candidat ingénieur
Modèle Bedrock	mistral.mistral-large-3-675b-instruct
Alias Codex	aws-mistral-large-3
Provider	aws-mistral-large-3
API locale	http://127.0.0.1:4002/v1
Codex Apps	OFF
Shell natif	ON

6. Le moment décisif



Capture de réussite : PowerShell confirme MISTRAL_OK, Codex Windows répond et commence à travailler sur le projet.

La différence avec les essais précédents est visible : **le 400 lié aux ToolSpecification a disparu**. Codex Windows reçoit une réponse, connaît le projet DELA-JEU et tente d'exécuter des commandes. Le candidat a donc réellement franchi la couche Codex Windows.

7. Le faux nom : « GPT-6-Sol »

Lorsqu'on lui demande son modèle, la réponse affichée affirme être « GPT-6-Sol ». Cette phrase ne constitue pas une preuve de routage. Le test préalable du bridge avait déjà confirmé **MISTRAL_OK** sur le port 4002 et la configuration Codex pointait vers l'alias Mistral. L'interprétation opérationnelle est que le modèle reprend une identité ou un contexte injecté par l'environnement Codex, alors que le backend testé reste Mistral Large 3.

Règle de vérification : pour identifier le backend, faire confiance au routage et au test du bridge, pas à l'auto-identification textuelle du modèle.

8. Nouveau problème découvert : Windows n'est pas Unix

Une fois les outils acceptés, Mistral a tenté plusieurs commandes mais a utilisé une syntaxe POSIX avec `&&` dans un shell PowerShell. C'est un problème de comportement de l'agent, pas d'infrastructure. Le shell est donc bien accessible ; le candidat doit simplement apprendre que son environnement d'exécution est **Windows PowerShell**.

9. Procédure de démarrage validée

La procédure ONE-CLICK finale doit conserver les invariants suivants :

01	Ne jamais toucher au bridge Kimi K3 / port 4001.
02	Vérifier/ajouter Mistral Large 3 dans config.yaml.
03	Configurer Codex Windows sur aws-mistral-large-3.
04	Désactiver Apps, remote plugins et multi-agent pour éviter les noms MCP incompatibles AWS.
05	Conserver shell_tool et shell_snapshot.
06	Démarrer LiteLLM sur 4002.
07	Tester un vrai appel et exiger MISTRAL_OK.
08	Fermer uniquement l'ancienne application Codex Windows.
09	Ouvrir l'application OpenAI.Codex Windows.
10	Créer un nouveau chat afin de ne pas réutiliser un ancien catalogue d'outils.

10. État du recrutement

Mistral Large 3 n'est pas encore « embauché » comme ingénieur définitif : il doit maintenant être jugé sur son comportement dans le vrai projet. Mais l'obstacle le plus important est franchi : **il fonctionne réellement derrière Codex Windows sur AWS Bedrock.**

La prochaine évaluation ne doit pas être une série de benchmarks artificiels. Elle doit mesurer ce qui compte pour le poste : compréhension du dépôt, prudence avant modification, capacité à utiliser correctement PowerShell, qualité des décisions techniques, coordination avec Kimi K3 et respect de l'existant.

FRANKENSTEIN - ÉQUIPE EN FORMATION

PRODUCTEUR - décision finale

MISTRAL LARGE 3 - candidat ingénieur / Codex Windows / 4002

KIMI K3 - codeur / terminal / 4001

GODOT - terrain de vérité

« Le cerveau est entré dans le bureau. Maintenant, il doit prouver qu'il sait diriger l'atelier. »